Phage SECURITY

Spiral Stake

Security Review

Conducted by:

Pyro, Security Researcher
YanecaB, Security Researcher
yovchev_yoan, Security Researcher

19.01.2026

Table of Contents

About Phage Security . . . 3
Disclaimer . . . 3
System overview . . . 3
Executive summary . . . 3
- Overview . . . 3
- Timeline . . . 4
- Scope . . . 4
- Issues Found . . . 4
- Findings Summary . . . 5

Findings . . . 6
- High Severity . . . 6
  - [H-01] `deleverage` reverts on liquidated positions due to stale cached data . . . 6
- Medium Severity . . . 7
  - [M-01] Flash loan surplus becomes stuck when external actor repays user debt . . . 7
  - [M-02] `deleverage` reverts due to insolvency when swap proceeds are insufficient . . . 9
  - [M-03] Stuck rewards in `UserProxy` . . . 9
  - [M-04] Creating positions exceeding maxLTV . . . 10
- Low Severity . . . 11
  - [L-01] Repaying or supplying positions directly with Morpho increases the yield (and thus the yield fee) . . . 11
  - [L-02] The yield fee also takes a fee on collateral value increases . . . 12
  - [L-03] Recovery mode cannot manage positions due to missing token approvals . . . 13

About Phage Security

Phage Security is a team of highly skilled smart contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike. With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

Check out our previous work at PhageSecurity.com or reach out on X to Pyro.

Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

Spiral Stake is a leveraged yield protocol that uses flash loans to create amplified stablecoin positions in a single atomic transaction—replacing dozens of manual deposit-borrow loops with one optimized operation. Users deposit collateral, and the protocol flash-borrows the optimal amount from Morpho, supplies the combined capital, borrows against it to repay the flash loan, leaving users with a fully leveraged position they can manage (increase leverage, supply more collateral, repay debt, or close).

Executive summary

Overview

Project Name	Spiral Stake
Repository	private
Commit hash	b15fbbffa0ff6abef270a630c800eade6e477211
Remediation	b9d60a0b20c6488ae65981714659c443b0318ee9
Methods	Manual review

Timeline

Audit kick-off	09.01.2026
End of audit	12.01.2026
Remediations start	13.01.2026
Remediations end	18.01.2026

Scope

core/FlashLeverage/FlashLeverage.sol

core/FlashLeverage/MarketPositionManager.sol

core/FlashLeverage/SwapManager.sol

core/FlashLeverage/UserProxy.sol

core/libraries/TokenHelper.sol

core/libraries/Math.sol

Issues Found

Severity	Count
High	1
Medium	4
Low	3

Findings Summary

ID	Title	Severity	Status
[H-01]	‘deleverage‘ reverts on liquidated positions due to stale cached data	High	Fixed
[M-01]	Flash loan surplus becomes stuck when external actor repays user debt	Medium	Fixed
[M-02]	‘deleverage‘ reverts due to insolvency when swap proceeds are insufficient	Medium	Fixed
[M-03]	Stuck rewards in ‘UserProxy‘	Medium	Fixed
[M-04]	Creating positions exceeding maxLTV	Medium	Fixed
[L-01]	Repaying or supplying positions directly with Morpho increases the yield (and thus the yield fee)	Low	Fixed
[L-02]	The yield fee also takes a fee on collateral value increases	Low	Fixed
[L-03]	Recovery mode cannot manage positions due to missing token approvals	Low	Acknowledged

Findings

High Severity

[H-01] `deleverage` reverts on liquidated positions due to stale cached data

`FlashLeverage` relies on cached values in the `LeveragePosition` struct (`sharesBorrowed` and `amountLeveragedCollateral`) to execute the deleveraging process. However, these values become stale if a position is partially or fully liquidated on Morpho, as the liquidation logic on Morpho reduces the user's debt and collateral balance without updating the `FlashLeverage` state.

1. A user has a position with `10 ETH` collateral and `20 000 USDC` debt. Due to a price drop, a liquidator seizes `5 ETH` and repays `10 000 USDC`.

- Actual Morpho state: `5 ETH` collateral, `10 000 USDC` debt.
- Cached contract state: `10 ETH` collateral, `20 000 USDC` debt.

2. The user attempts to close the remaining position. The function reads the cached `position.amountLeveragedCollateral` (`10 ETH`).
3. The contract calls `_repayAndWithdrawCollateralViaProxy` using the cached `10 ETH`.

```
function _repayAndWithdrawCollateralViaProxy(...) internal {
...
    if (amountCollateral > 0) {
       _morphoWithdrawCollateralViaProxy(
          userProxy,
          marketParams,
          amountCollateral
       );
    }
```

4. The `UserProxy` forwards this call to Morpho. Morpho checks the user's balance (`5 ETH`) against the requested withdrawal (`10 ETH`) and reverts.

Recommendation

Update the `deleverage` function to synchronize the `LeveragePosition` storage with live Morpho data before initiating the flash loan, ensuring the contract uses accurate collateral and debt values even after external liquidations.

Medium Severity

[M-01] Flash loan surplus becomes stuck when external actor repays user debt

When an external actor repays part of a user's Morpho debt before `deleverage()` is called, the flash loan amount is calculated using stale cached data while the actual debt repayment uses live Morpho data. This mismatch causes surplus flash loan funds to become permanently stuck in the contract.

The flash loan is repaid from the swapped collateral. When the flash loan amount exceeds what was actually needed for debt repayment, the surplus remains in the contract but is not accounted for in the distribution calculation.

Root Cause:

The flash loan amount is calculated using the cached `position.sharesBorrowed`:

```
uint256 amountFlashLoan = calcDeleverageFlashLoan(
   position.collateralToken,
   position.loanToken,
   position.sharesBorrowed  // Cached value (e.g., 16,000 shares)
);
```

However, `_morphoRepay()` reads the actual current debt from Morpho:

```
uint256 borrowSharesLeft = i_morpho.position(...).borrowShares;  // Actual
   value (e.g., 15,900 shares)

sharesBorrowed = borrowSharesLeft < sharesBorrowed
   ? borrowSharesLeft
   : sharesBorrowed;
```

This creates a surplus: - Flash loan: 16,160 USDC (based on 16,000 cached shares) - Actual repayment: 16,060 USDC (based on 15,900 actual shares) - Surplus: 100 USDC remains in contract

The distribution calculation ignores this surplus:

```
totalAmountReturned = amountSwappedLoanToken - amountLoan;
// Only accounts for swap output minus flash loan
// Doesn't include the unused flash loan portion
```

Detailed Example:

Initial State: - User position: 4.999 wstETH collateral, 16,000 borrow shares - FlashLeverage cache: `sharesBorrowed = 16,000`

External Repayment: - External actor repays 100 USDC of user's debt - Morpho state: `borrowShares = 15,900` (reduced) - FlashLeverage cache: `sharesBorrowed = 16,000` (stale)

User Calls `deleverage()`:

1. **Calculate flash loan** (uses cached 16,000 shares):
   - Flash loan amount: 16,160 USDC
   - Contract receives: 16,160 USDC
2. **Repay debt** (uses actual 15,900 shares):
   - Morpho pulls: 16,060 USDC
   - Remaining in contract: 100 USDC
3. **Withdraw and swap collateral**:
   - Withdraw:

4.999 wstETH - Swap to USDC: 20,480 USDC - Total in contract: 100 + 20,480 = 20,580 USDC

4. **Repay flash loan** (from swapped collateral):
   - Flash loan repayment: 16,160 USDC
   - Remaining: 20,580 - 16,160 = 4,420 USDC
5. **Calculate distribution** (bug occurs here):

```
totalAmountReturned = amountSwappedLoanToken - amountLoan
                    = 20,480 - 16,160
                    = 4,320 USDC
```

   Actual balance: 4,420 USDC Calculated: 4,320 USDC Discrepancy: 100 USDC
6. **Distribute funds**:
   - Fee (10%): 32 USDC to treasury
   - User: 4,288 USDC
   - Total distributed: 4,320 USDC
   - **Stuck in contract: 100 USDC**

Recommendation

Sync `position.sharesBorrowed` with the actual current debt from Morpho before calculating the flash loan amount in `deleverage()`. This ensures the flash loan matches the actual debt to be repaid, preventing surplus funds from becoming stuck in the contract.

This fix resolves the stuck funds issue but will include any externally repaid debt amounts in the yield calculation, causing the protocol to charge fees on these "gifts." If the protocol's intended fee model is to only charge on yield generated by the leveraged position, additional logic is needed.

[M-02] `deleverage` reverts due to insolvency when swap proceeds are insufficient

The `deleverage` function executes a flash loan to repay a user's debt on Morpho, withdraws the collateral, and swaps it for the loan token to repay the flash loan. The logic assumes that the proceeds from the collateral swap (`amountSwappedLoanToken`) will always be sufficient to cover the flash loan debt (`amountLoan`).

```
// Swap withdrawn collateral -> loan token
    uint256 amountSwappedLoanToken = _swapToken(
       position.collateralToken,
       position.loanToken,
       position.amountLeveragedCollateral,
       swapData,
       minTokenOut
    );

    // Repay the flash loan, with swapped loan token
    _forceApprove(position.loanToken, address(i_morpho), amountLoan);

    uint256 totalAmountReturned;
    if (amountSwappedLoanToken > amountLoan) {
       unchecked {
          totalAmountReturned = amountSwappedLoanToken - amountLoan;
       }
    }
```

However, if the position is underwater (`collateral value < debt`) or if slippage and fees reduce the swap output below the debt amount, the contract will hold insufficient funds to repay the flash loan.

This creates a DoS condition where users with deteriorating positions are "trapped." They cannot close their positions to stop losses because the `deleverage` function consistently reverts, forcing them to wait for liquidation.

Recommendation

Modify `_handleDeleverage` to calculate the shortfall if the swap proceeds are less than the loan amount. Use `_transferIn` to pull the remaining required funds directly from the user's wallet. This ensures the flash loan is always repaid and allows users to successfully close positions even when they are at a loss.

[M-03] Stuck rewards in `UserProxy`

The `UserProxy` is designed to isolate user positions, but its `execute` function is hardcoded to only allow calls to the `morpho` contract. In the Morpho ecosystem, rewards (such as MORPHO tokens) are often distributed via separate contracts and accrue directly to the position holder (in our case the `UserProxy`).

Because the proxy cannot execute calls to token contracts (e.g. `ERC20.transfer`)

```
function execute(
   bytes calldata data
) external returns (bytes memory result) {
   if (msg.sender == flashLeverage) {
      // Proceed
   } else if (msg.sender == user) {
      require(recoveryMode, "UserProxy: Not in Recovery Mode");
      // Proceed
   } else {
      revert("UserProxy: Unauthorised");
   }

   (bool success, bytes memory returnData) = morpho.call(data);
   require(success, "UserProxy: Call Failed");
   return returnData;
}
```

these rewards, are permanently locked.

Morpho Docs: https://docs.morpho.org/learn/concepts/rewards/

Recommendation

Implement the reward claiming functionality so that the reward can be claimed whenever they become available.

[M-04] Creating positions exceeding maxLTV

When users create leveraged positions, the protocol calculates a flash loan amount based on the desired LTV using oracle prices. However, the actual swap from loan token to collateral token may result in fewer tokens than expected due to slippage. Since the protocol does not validate the final LTV after the swap completes, positions can be created with an actual LTV that exceeds the configured maxLTV.

The flash loan calculation assumes perfect swap execution at oracle prices:

```
// Assumes 1:1 swap at oracle price
uint256 amountLoan = totalPositionValue - collateralValue;
```

But the actual swap may return less collateral:

```
// User gets less PT than expected due to slippage
amountSwappedCollateral = _swapTokenToPt(...);
// No validation that final position LTV <= maxLTV
```

Recommendation

Add LTV validation after swap execution in `_handleLeverage()`.

Low Severity

[L-01] Repaying or supplying positions directly with Morpho increases the yield (and thus the yield fee)

When repaying positions, the protocol increases `amountDepositedInLoanToken`:

```
_morphoRepay(
   position.userProxy,
   s_marketParams[collateralToken][loanToken],
   amountRepay,
   0 // Not repaying by shares
);
position.amountDepositedInLoanToken += amountRepay;
```

This variable is used to calculate yield by subtracting the loan (`amountLoan`) and the supplied collateral (`amountDepositedInLoanToken`) from the total position value (`amountSwappedLoanToken`):

```
if (amountSwappedLoanToken > amountLoan) {
   unchecked {
      totalAmountReturned = amountSwappedLoanToken - amountLoan;
   }
}
if (s_isCorrelated[position.collateralToken][position.loanToken]) {
   uint256 amountDeposited = position.amountDepositedInLoanToken;
   // ...
   if (totalAmountReturned > amountDeposited) {
      uint256 yieldGenerated = totalAmountReturned - amountDeposited;
```

However, users can repay positions directly through Morpho on behalf of their proxy contracts. This bypasses the protocol logic, meaning `amountDepositedInLoanToken` is not increased. Consequently, the yield calculation will incorrectly treat the repayment amount as profit.

The same happens if the user supplies collateral for the proxy using Morpho.

Recommendation

Since a dedicated `repay` function exists, this action can be considered user error. It is recommended to document this behavior and advise users against interacting with their positions directly through Morpho.

[L-02] The yield fee also takes a fee on collateral value increases

When creating a position, the protocol calculates `amountDepositedInLoanToken` as the value of collateral tokens in terms of the loan token.

```
uint256 amountDepositedInLoanToken = getCollateralValueInLoanToken(
   collateralToken,
   loanToken,
   amountCollateral
).scaleTo(Math.STANDARD_DECIMALS, IERC20Metadata(loanToken).decimals());
```

When closing the position, the value of the returned collateral is determined by subtracting the loan amount from the total swapped position value:

```
uint256 totalAmountReturned;
if (amountSwappedLoanToken > amountLoan) {
   unchecked {
      totalAmountReturned = amountSwappedLoanToken - amountLoan;
   }
}
```

Interest earned is then calculated by subtracting the initial collateral value from the total returned amount:

```
if (totalAmountReturned > amountDeposited) {
   uint256 yieldGenerated = totalAmountReturned - amountDeposited;
```

This process fails to account for collateral price appreciation. If the value of the collateral token increases relative to the loan token, the protocol incorrectly identifies the price gain on the user's initial deposit as yield and charges a performance fee on it.

Example:

- Collateral token: wETH (4000 USD)
- Loan token: USDC (1 USD)

Initial Position:

1. Collateral - 10 wETH (40k USD)
2. Leveraged position -15 wETH (60k USD)

3. Loan - 20k USDC
4. `amountDepositedInLoanToken` - 40k USDC

Closing position (wETH price increases to 5000 USD):

1. The 15 wETH position is swapped for 75k USDC
2. The 20k USDC loan is repaid
3. `totalAmountReturned` is 55k USDC
4. The protocol calculates yield as 55k - 40k = 15k USDC
5. A 10% fee is charged on the 15k USDC

The fee is inaccurate because the whole increase is simply the price appreciation of the position, not profit generated by yield.

The same will happen in reverse. If yield is generated, but the collateral token looses value compared to the loan token, then the yield is not taxed and thus the project earns less.

Recommendation

Consider tracking the coll in it's coll token amounts and charge a yield on that increase. In this case it would have been to track 10 wETH and charge a 10% fee if that wETH has increased.

This way price will not play a factor.

[L-03] Recovery mode cannot manage positions due to missing token approvals

Recovery mode is intended to allow users to manually manage their positions after issues occur (such as partial liquidations). However, UserProxy cannot execute the necessary token approvals for Morpho operations, making recovery mode non-functional for critical actions.

Root Cause:

UserProxy's `execute()` function is hardcoded to only call the Morpho contract:

```
function execute(bytes calldata data) external returns (bytes memory result) {
    if (msg.sender == flashLeverage) {
        // Proceed
    } else if (msg.sender == user) {
        require(recoveryMode, "UserProxy: Not in Recovery Mode");
        // Proceed
    } else {
        revert("UserProxy: Unauthorised");
    }

    (bool success, bytes memory returnData) = morpho.call(data); // Can only
        call morpho
    require(success, "UserProxy: Call Failed");
    return returnData;
}
```

When users attempt to supply collateral or repay debt via recovery mode:

1. User transfers tokens to UserProxy
2. User calls `UserProxy.execute(supplyCollateral(...))` or `UserProxy.execute(repay(...))`
3. UserProxy forwards call to Morpho
4. Morpho attempts `token.transferFrom(UserProxy, Morpho, amount)`
5. Transfer fails because `token.allowance(UserProxy, Morpho) = 0`

UserProxy has no mechanism to approve tokens because:

- `execute()` can only call Morpho, not token contracts
- No separate function exists to set approvals
- No callback implementations to set approvals during Morpho operations

Scenario:

1. Position gets partially liquidated (some collateral seized, some debt repaid)
2. User wants to repay remaining debt and withdraw remaining collateral
3. Cannot repay via recovery mode (no USDC approval to Morpho)
4. Cannot withdraw collateral (Morpho requires debt to be repaid first)
5. Cannot use `deleverage()` (reverts due to stale cached collateral amount)

Recommendation

Allow the target contract to be specified by the user.